

FPGA BASED SMART TRAFFIC LIGHT CONTROLLER

Project Report

M. V. S. Charith

Department of Electronics and Communication Engineering

Indian Institute of Technology, Kharagpur.

ABSTRACT

This project presents the design and FPGA-oriented implementation of a Smart Traffic Light Controller using Verilog HDL and Xilinx Vivado. The system was designed using a Finite State Machine (FSM) architecture capable of handling multiple real-world traffic control scenarios including normal traffic sequencing, pedestrian crossing requests, emergency vehicle prioritization, and night mode blinking operation.

The controller was implemented using modular Verilog design methodology consisting of separate modules for FSM control, timer generation, clock enable generation, and top-level integration. A configurable timer-based approach was used to control traffic durations and state transitions efficiently.

The project also focused on complete FPGA development workflow including behavioral simulation, waveform verification, synthesis, implementation, routing, timing analysis, constraint handling, and successful bitstream generation targeting an Artix-7 FPGA device.

Comprehensive testbench verification was performed for various operating conditions such as:

- reset behavior,
- pedestrian requests,
- emergency overrides,
- simultaneous request prioritization,

- night mode transitions,
- rapid mode toggling.

The project provided practical understanding of FPGA-based digital system design, FSM implementation, verification methodology, hardware constraints, and Vivado implementation flow. The final design successfully demonstrated stable operation and correct state transitions under all verified test conditions.

PROJECT SUMMARY

Parameter	Description
HDL Used	Verilog HDL
FPGA Tool	Xilinx Vivado 2024.x
FPGA Family	Artix-7
Design Methodology	FSM-Based Modular Design
Verification Method	Behavioral Simulation
FPGA Flow	Synthesis → Implementation → Bitstream
Emergency Handling	Supported
Pedestrian Support	Supported
Night Mode	Supported
Bitstream Generation	Successful

INTRODUCTION

Traffic management systems are important components of modern urban infrastructure. Conventional traffic signal systems often operate using fixed timing mechanisms without dynamically handling pedestrian crossings, emergency vehicle

prioritization, or reduced-traffic night operation. Such limitations may reduce traffic efficiency and affect road safety.

With the advancement of digital hardware systems, FPGA-based traffic controllers provide a flexible and reconfigurable approach for implementing intelligent traffic management solutions. Field Programmable Gate Arrays (FPGAs) enable parallel processing, modular design, rapid prototyping, and real-time control, making them suitable for traffic control applications.

In this project, a Smart Traffic Light Controller was designed and implemented using Verilog HDL on an FPGA-oriented workflow. The system was developed using a Finite State Machine (FSM) architecture capable of handling:

- normal traffic sequencing,
- pedestrian crossing requests,
- emergency vehicle override conditions,
- night mode blinking operation.

The design was verified through behavioral simulation and later synthesized and implemented using Xilinx Vivado. FPGA implementation stages including synthesis, placement, routing, timing verification, constraint assignment, and successful bitstream generation were completed successfully.

The project emphasizes modular RTL design, verification methodology, practical FPGA implementation flow, and real-world digital system considerations.

PROJECT OBJECTIVES

The primary objective of this project was to design and implement an FPGA-based Smart Traffic Light Controller using Verilog HDL and Xilinx Vivado.

The project aimed to demonstrate practical FPGA-oriented digital system design using a modular Finite State Machine (FSM) architecture capable of handling multiple traffic management scenarios.

The major objectives of the project are:

- To design a modular traffic light controller using Verilog HDL
- To implement FSM-based traffic sequencing logic
- To handle pedestrian crossing requests safely
- To implement emergency vehicle priority override operation
- To design night mode blinking functionality for low-traffic conditions
- To implement timer-controlled state transitions

- To perform behavioral simulation and waveform verification
- To understand FPGA synthesis and implementation workflow
- To perform timing analysis and resource utilization analysis
- To implement FPGA constraints and physical pin mapping
- To successfully generate FPGA bitstream after implementation
- To gain practical exposure to debugging and FPGA design-flow verification

The project also focused on improving understanding of real-world digital control systems and FPGA-based embedded design methodologies.

PROBLEM STATEMENT

Conventional traffic light systems commonly operate using fixed sequential timing mechanisms without considering dynamic road conditions or special operational requirements such as pedestrian crossings, emergency vehicle access, or reduced night-time traffic.

In practical traffic management systems, the following challenges are frequently encountered:

- Lack of pedestrian crossing support
- Delayed emergency vehicle movement
- Inefficient night-time operation
- Limited adaptability of fixed-timing systems
- Difficulty in modifying hardware-based control logic

Traditional hardware implementations using discrete logic components may increase circuit complexity and reduce scalability. FPGA-based solutions provide a flexible and reconfigurable alternative capable of implementing intelligent control systems using programmable digital logic.

The objective of this project is to design and verify a Smart Traffic Light Controller capable of:

- managing normal traffic sequencing,
- handling pedestrian crossing requests,
- prioritizing emergency vehicle movement,
- supporting night mode blinking operation,
- performing stable FSM-based state transitions.

The system should also support complete FPGA implementation flow including simulation, synthesis, routing, timing verification, physical pin mapping, and bitstream generation.

FPGA AND VERILOG BACKGROUND

Field Programmable Gate Array (FPGA)

A Field Programmable Gate Array (FPGA) is a reconfigurable semiconductor device consisting of programmable logic blocks, configurable interconnections, routing resources, and input/output peripherals. FPGA devices are widely used in digital system design because they allow hardware functionality to be modified through programmable logic implementation.

FPGA devices are commonly used in:

- embedded systems,
- communication systems,
- digital signal processing,
- control systems,
- real-time hardware applications,
- rapid prototyping.

The general FPGA development flow includes:

1. HDL Design Entry
2. Behavioral Simulation
3. Synthesis
4. Implementation
5. Timing Verification
6. Bitstream Generation
7. Hardware Deployment

In this project, an Artix-7 FPGA-oriented workflow was followed using Xilinx Vivado Design Suite.

Verilog Hardware Description Language

Verilog HDL is a hardware description language used for modeling and designing digital systems. It supports both combinational and sequential circuit design and enables modular hardware implementation.

Verilog allows designers to:

- describe digital hardware behavior,
- model FSM-based systems,
- design synchronous sequential circuits,

- perform simulation and verification,
- implement modular architectures.

In this project, Verilog HDL was used to implement:

- FSM-based traffic control logic,
- timer counter module,
- clock enable generation,
- request handling logic,
- top-level module integration,
- behavioral testbench verification.

The modular Verilog approach improved readability, scalability, debugging efficiency, and FPGA implementation management.

TOOLS AND TECHNOLOGIES USED

- Verilog HDL
- Xilinx Vivado Design Suite
- Artix-7 FPGA
- Behavioral Simulation
- FPGA Synthesis and Implementation

PROPOSED SYSTEM

The proposed system is an FPGA-based Smart Traffic Light Controller implemented using Verilog HDL and a Finite State Machine (FSM) architecture.

The controller manages traffic flow between a main road and a side road while supporting additional intelligent control features including pedestrian crossing requests, emergency vehicle handling, and night mode operation.

The system accepts the following inputs:

Input Signal	Description
clk	System clock input

reset	System reset signal
pedestrian_request	Pedestrian crossing request input
emergency_request	Emergency vehicle request input
night_mode	Night mode enable input

The system generates the following outputs:

Output Signal	Description
main_traffic_lights	Main road traffic light control
side_traffic_lights	Side road traffic light control
Time	Countdown timer output
ped_crossing_enable	Pedestrian crossing indicator

The traffic controller operates using multiple FSM states including:

- MAIN_GREEN
- MAIN_YELLOW
- ALL_RED_1
- SIDE_GREEN
- SIDE_YELLOW
- ALL_RED_2
- PEDESTRIAN_CROSSING

- EMERGENCY
- NIGHT_MODE_HIGH
- NIGHT_MODE_LOW

The architecture includes:

- FSM control module
- timer counter module
- clock enable generator
- top-level integration module
- behavioral verification testbench

The system was designed using modular RTL methodology to simplify debugging, verification, scalability, and FPGA implementation.

SYSTEM ARCHITECTURE

The Smart Traffic Light Controller was designed using a modular FPGA-oriented architecture to simplify verification, debugging, scalability, and implementation.

The complete system consists of the following modules:

1. Traffic FSM Module
2. Timer Counter Module
3. Clock Enable Generator Module
4. Top-Level Integration Module
5. Testbench Module

The FSM module acts as the central control unit of the system and determines traffic light operation based on timer completion and external request inputs.

The timer counter module generates countdown-based timing control for state transitions. A clock enable generator is used to generate slower timing pulses from the system clock for traffic-duration control.

The top-level module integrates all submodules and connects internal control signals between the FSM and timer modules.

The overall architecture follows synchronous digital design methodology using a single system clock.

Internal Functional Blocks

Module	Function
traffic_fsm	Controls state transitions and traffic logic
timer_counter	Generates countdown timing for each state
clock_enable_generator	Produces slower timing tick pulses
top_traffic_controller	Integrates all modules
traffic_controller_tb	Performs behavioral verification

Block Diagram

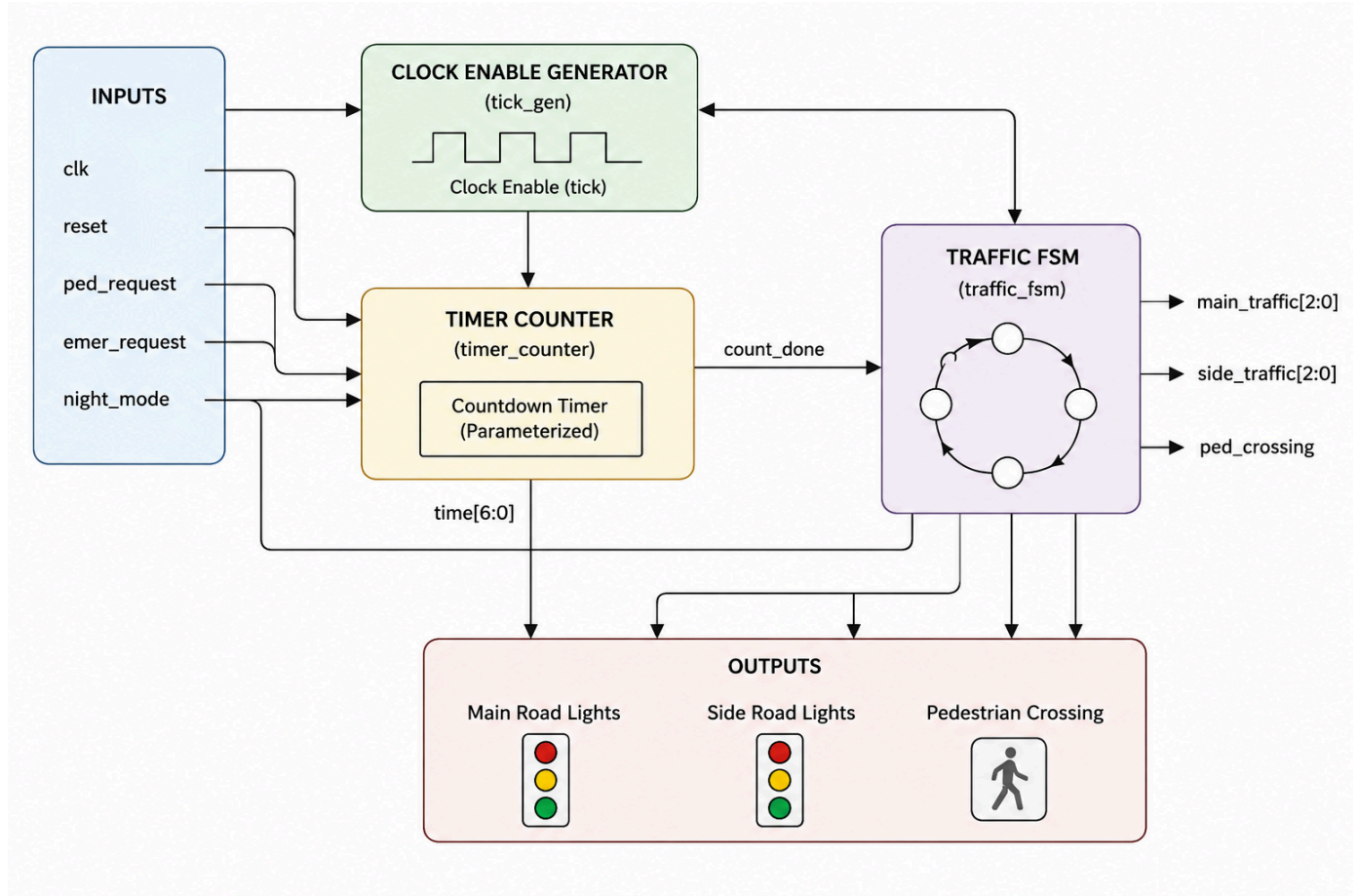


Figure 1: Block Diagram of FPGA Based Smart Traffic Light Controller

FSM DESIGN AND STATE DESCRIPTION

The Smart Traffic Light Controller was implemented using a Finite State Machine (FSM) architecture. The FSM controls traffic signal sequencing, pedestrian handling, emergency prioritization, and night mode operation through multiple synchronous states.

The FSM design improves modularity, simplifies state transition management, and enables reliable sequential operation.

FSM States

The implemented FSM contains the following states:

State	Description
MAIN_GREEN	Main road green signal active
MAIN_YELLOW	Main road yellow transition
ALL_RED_1	Temporary all-red safety delay
SIDE_GREEN	Side road green signal active
SIDE_YELLOW	Side road yellow transition
ALL_RED_2	Temporary all-red safety delay
PEDESTRIAN_CROSSING	Pedestrian crossing enabled
EMERGENCY	Emergency vehicle priority mode
NIGHT_MODE_HIGH	Night mode blinking ON state
NIGHT_MODE_LOW	Night mode blinking OFF state

Normal Traffic Flow

During normal operation, the FSM follows this sequence:

MAIN_GREEN → MAIN_YELLOW → ALL_RED_1 → SIDE_GREEN → SIDE_YELLOW → ALL_RED_2 → MAIN_GREEN

Temporary ALL_RED states were added to improve traffic safety during signal transitions.

Pedestrian Crossing Operation

When a pedestrian request is detected:

- the request is latched internally,
- the FSM waits for a safe transition point,
- pedestrian crossing is enabled during the PEDESTRIAN_CROSSING state,
- traffic signals remain stopped during crossing operation.

This prevents unsafe interruption of active traffic flow.

Emergency Vehicle Handling

Emergency requests are given higher priority than pedestrian requests and normal traffic sequencing.

The FSM:

- safely transitions traffic signals,
- activates emergency traffic routing,
- resumes normal operation after emergency completion.

Special return-state handling was implemented to ensure safe recovery after emergency operation.

Night Mode Operation

Night mode was implemented using alternating blinking states:

- NIGHT_MODE_HIGH
- NIGHT_MODE_LOW

This produces blinking yellow-light behavior typically used during low-traffic conditions at night.

The blinking operation is controlled using timer-based state transitions.

Internal Request Registers

The FSM uses internal request registers to latch incoming requests:

Register	Purpose
ped_req_reg	Stores pedestrian requests
emer_req_reg	Stores emergency requests
prev_state	Stores previous FSM state
return_state	Stores safe return state

These registers improve stability and ensure correct request servicing behavior.

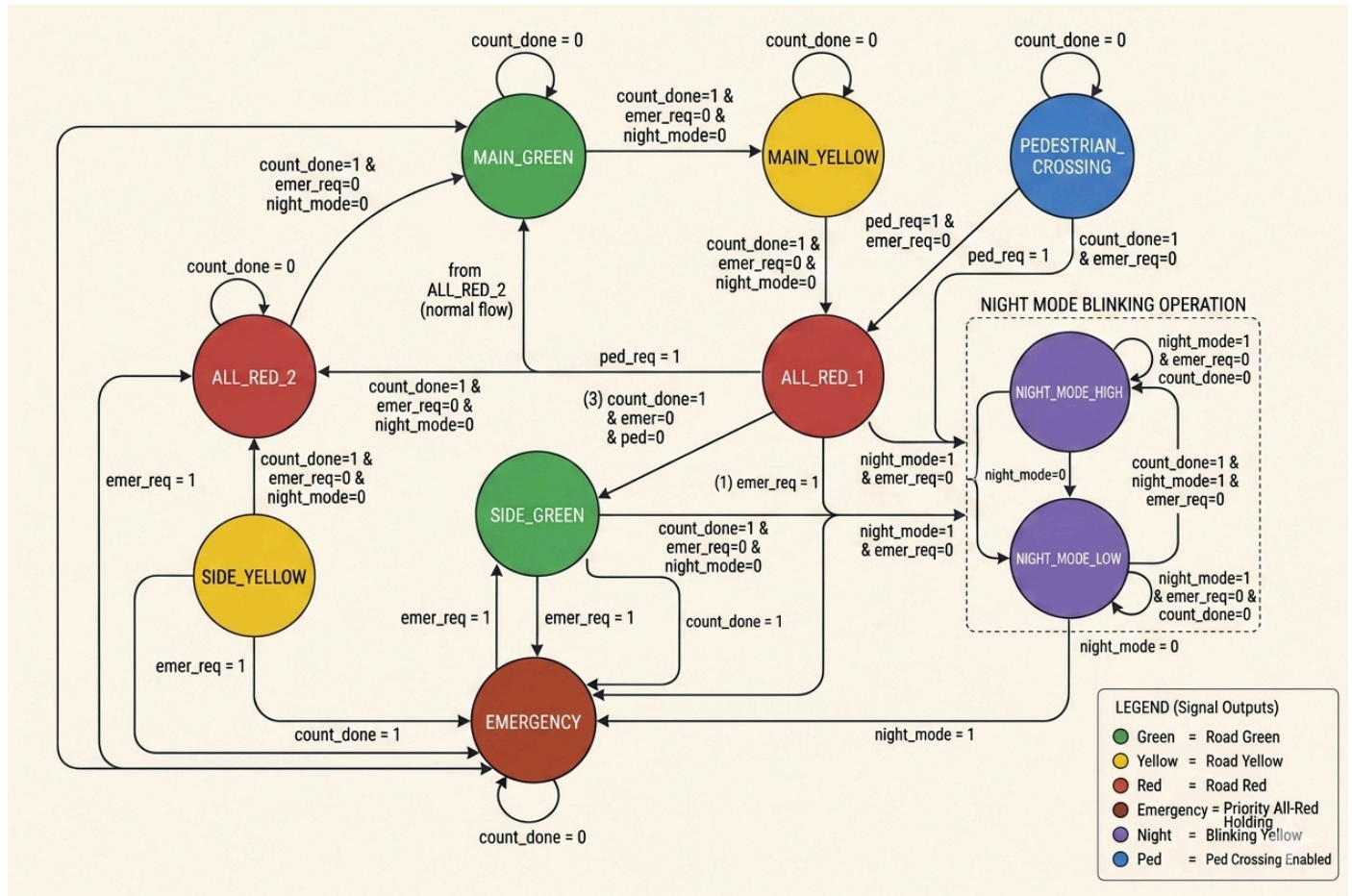


Figure 2.1: FSM State Transition Diagram of Smart Traffic Light Controller

START

↓

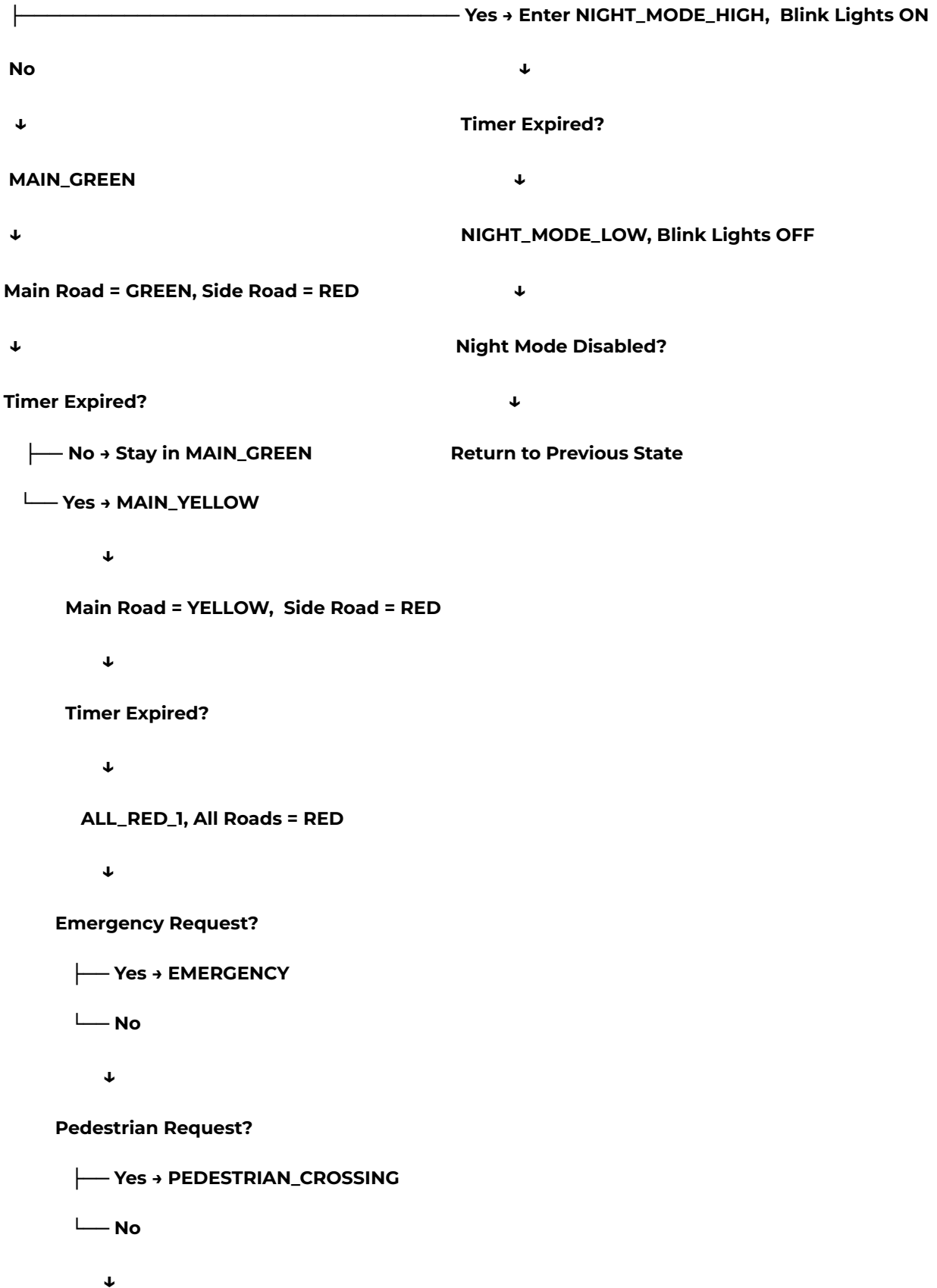
Initialize System, Reset All Registers and FSM States

↓

Set Initial State = ALL_RED_2 , Enable Timer Counter

↓

Check Night Mode?



SIDE_GREEN



Side Road = GREEN, Main Road = RED



Emergency Request?

|— Yes → **SIDE_YELLOW**

|— No



Timer Expired?

|— No → Stay in **SIDE_GREEN**

|— Yes → **SIDE_YELLOW**



Side Road = YELLOW, Main Road = RED



Timer Expired?



ALL_RED_2, All Roads = RED



Emergency Request?

|— Yes → **EMERGENCY**

|— No



Pedestrian Request?

|— Yes → **PEDESTRIAN_CROSSING**

|— No

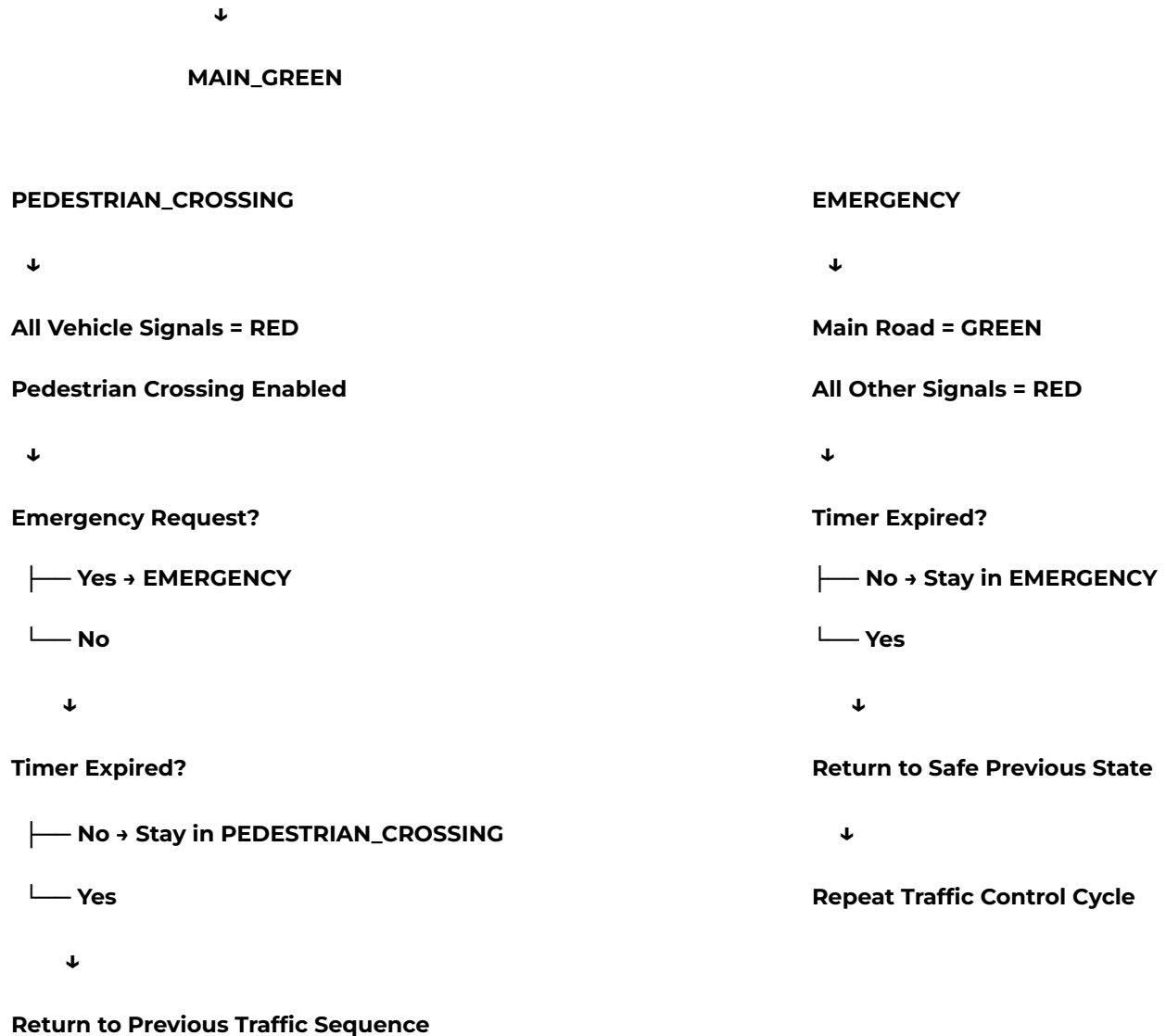


Figure 2.2: Operational Flowchart of Traffic Controller FSM

MODULE DESCRIPTION

The Smart Traffic Light Controller was implemented using multiple modular Verilog components. Each module performs a dedicated function within the complete traffic control system.

1. Traffic FSM Module

The traffic_fsm module acts as the primary control unit of the system. It controls traffic sequencing, request handling, timer control, and state transitions.

Major responsibilities include:

- FSM state transition control
- Traffic light signal generation
- Pedestrian request servicing
- Emergency vehicle handling
- Night mode blinking control
- Timer reset and duration control

The FSM was implemented using synchronous sequential logic and combinational next-state logic.

2. Timer Counter Module

The timer_counter module generates countdown timing for each FSM state.

Features include:

- configurable count duration,
- timer reset handling,
- tick-based counting,
- countdown-style output generation,
- count_done pulse generation.

The module increments only when the tick signal is active, allowing slower timing operation independent of the main system clock.

3. Clock Enable Generator Module

The clock_enable_generator module generates periodic tick pulses from the main FPGA system clock.

The module internally uses a programmable counter to divide the incoming clock frequency and generate slower enable pulses used by the timer module.

Different COUNT_STOP values were used for:

- fast behavioral simulation,
- FPGA-oriented implementation timing.

This significantly improved simulation speed while maintaining scalable hardware-oriented design.

4. Top-Level Integration Module

The `top_traffic_controller` module integrates all internal submodules.

This module connects:

- FSM control logic,
- timer counter,
- clock enable generator,
- external input/output ports.

The module also supports parameterized `COUNT_STOP` configuration for simulation and hardware scaling.

5. Testbench Module

The `traffic_controller_tb` module was developed for behavioral verification of the entire system.

The testbench verifies:

- reset operation,
- normal traffic sequencing,
- pedestrian request handling,
- emergency override operation,
- night mode operation,
- edge-case transitions,
- rapid mode toggling.

Waveform dumping and monitoring statements were used for debugging and verification analysis.

SIMULATION RESULTS AND VERIFICATION

Behavioral simulation was performed using Vivado Simulator and waveform analysis was carried out. Multiple test scenarios were developed to verify the functionality, robustness, and correctness of the FPGA Based Smart Traffic Light Controller under various traffic conditions.

The simulation verified the following functionalities:

- Correct FSM state transitions
- Timer-based traffic sequencing
- Pedestrian request servicing
- Emergency vehicle priority handling
- Night mode blinking operation
- Priority handling between simultaneous requests
- Edge-case handling at count_done boundaries
- Timer reset synchronization

The testbench was designed using multiple independent test scenarios which could be enabled individually for easier debugging and waveform analysis.

1. SIMULATION ENVIRONMENT

Parameter	Value
Simulation Tool and Waveform Viewer	Vivado Simulator
HDL Language	Verilog HDL
Clock Period	10 ns
Tick Generator Parameter	COUNT_STOP = 3
Waveform Format	VCD
Verification Method	Behavioral Simulation

2. TESTBENCH OVERVIEW

The module traffic_controller_tb.v was developed to generate the required clock, reset, and input stimulus signals for validating the Smart Traffic Light Controller under different operating conditions.

The testbench includes:

- Continuous clock generation
- Reset initialization
- VCD waveform dumping
- Real-time monitoring using \$monitor
- Independent commented test scenarios
- Controlled simulation termination

The following outputs were verified during simulation:

- Main road traffic light outputs
- Side road traffic light outputs
- Countdown timer output
- Pedestrian crossing enable signal
- Emergency request behavior
- Night mode operation

3. TEST CASE SUMMARY

Test Case No.	Test Scenario	Purpose
Test 1	Initial Reset	Verify FSM initialization
Test 2	Pedestrian Request During MAIN_GREEN	Verify pedestrian servicing
Test 3	Pedestrian Request at count_done Edge	Verify edge synchronization

Test 4	Emergency Request During SIDE_GREEN	Verify emergency priority
Test 5	Emergency + Pedestrian Together	Verify request priority handling
Test 6	Night Mode Enable	Verify blinking mode entry
Test 7	Requests During Night Mode	Verify request behavior in night mode
Test 8	Exit Night Mode	Verify restoration of previous state
Test 9	Rapid Night Mode Toggling	Verify FSM stability

4. INDIVIDUAL TEST CASE RESULTS

TEST CASE 1 – INITIAL RESET VERIFICATION

Objective: To verify correct FSM initialization after reset assertion.

Procedure:

- Reset signal asserted at simulation start
- Reset released after 20 ns
- FSM transitions monitored

Expected Result:

- FSM initializes to ALL_RED_2 state
- Safe traffic condition maintained
- Timer reset correctly initialized

Observed Result:

The FSM initialized correctly and entered the default safe state without undefined outputs.

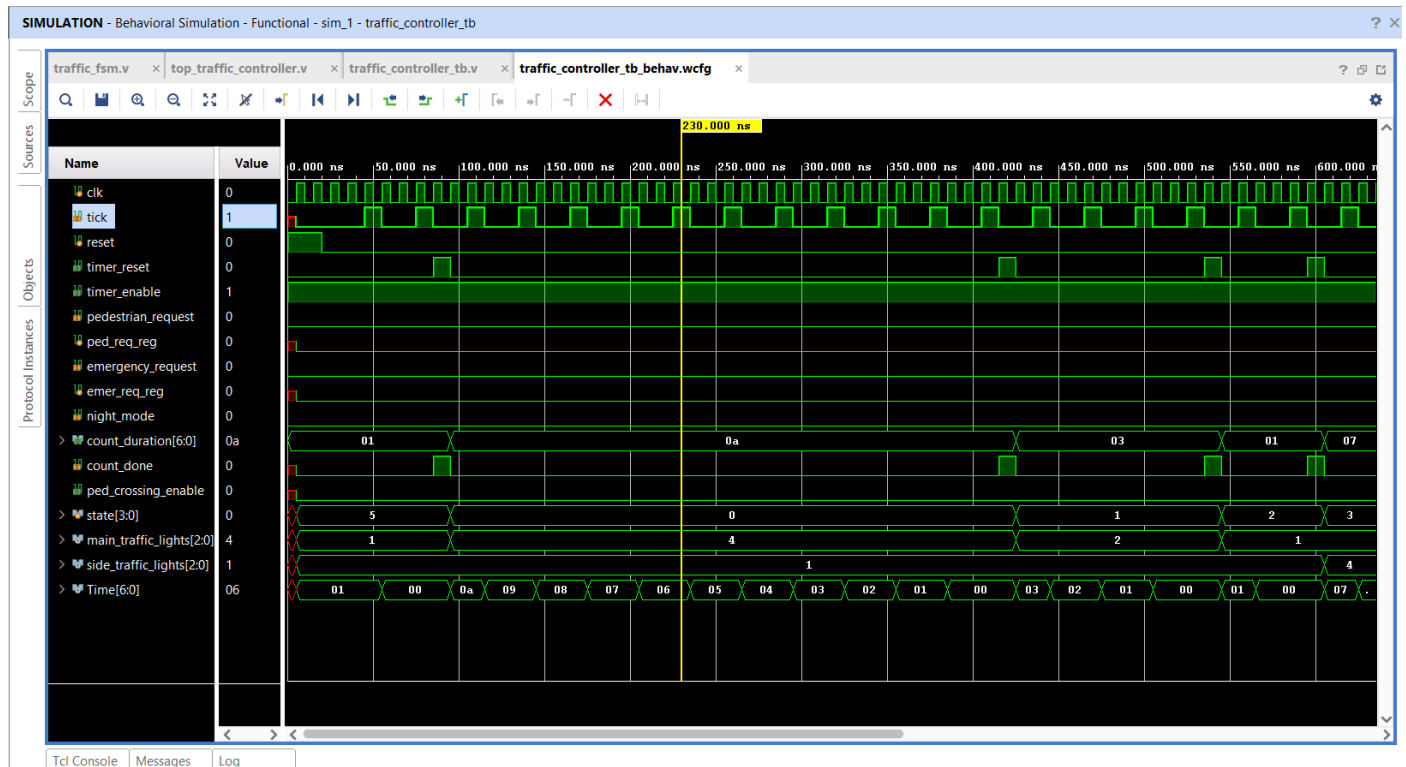


Figure 3.1 : Initial Reset Verification Waveform

TEST CASE 2 – PEDESTRIAN REQUEST DURING MAIN_GREEN

Objective: To verify pedestrian request latching and servicing.

Procedure:

- Pedestrian request asserted during MAIN_GREEN
- Request released after 10 ns

Expected Result:

- Request should latch internally
- Pedestrian crossing should occur during ALL_RED state

Observed Result:

The pedestrian request was successfully latched and serviced safely during the appropriate FSM transition.

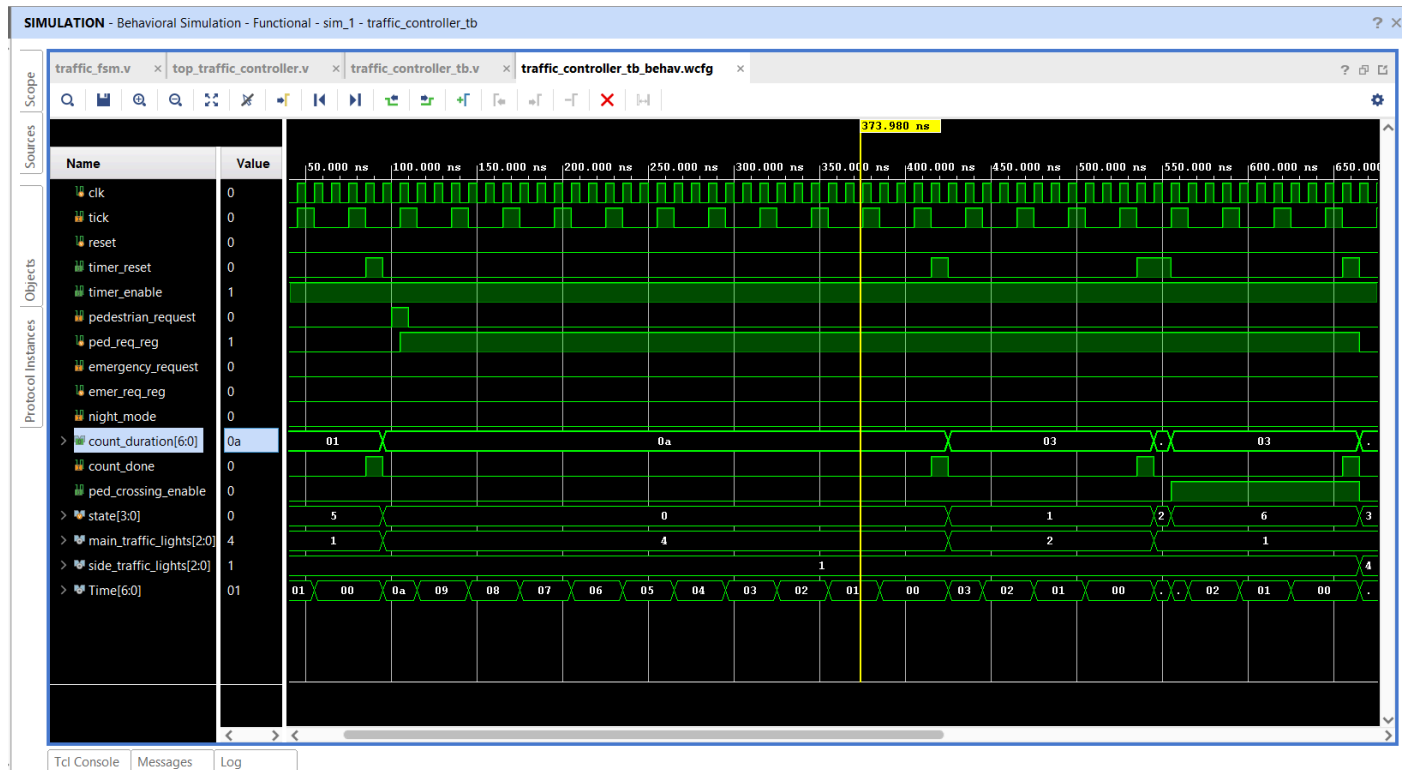


Figure 3.2 : Pedestrian Request Handling

TEST CASE 3 – PEDESTRIAN REQUEST AT count_done EDGE

Objective: To verify synchronization behavior at timer completion boundary.

Procedure:

- Pedestrian request asserted near count_done

Expected Result:

- No metastability or skipped transition
- Proper request servicing

Observed Result:

The FSM handled the edge-aligned request correctly without incorrect state transitions.

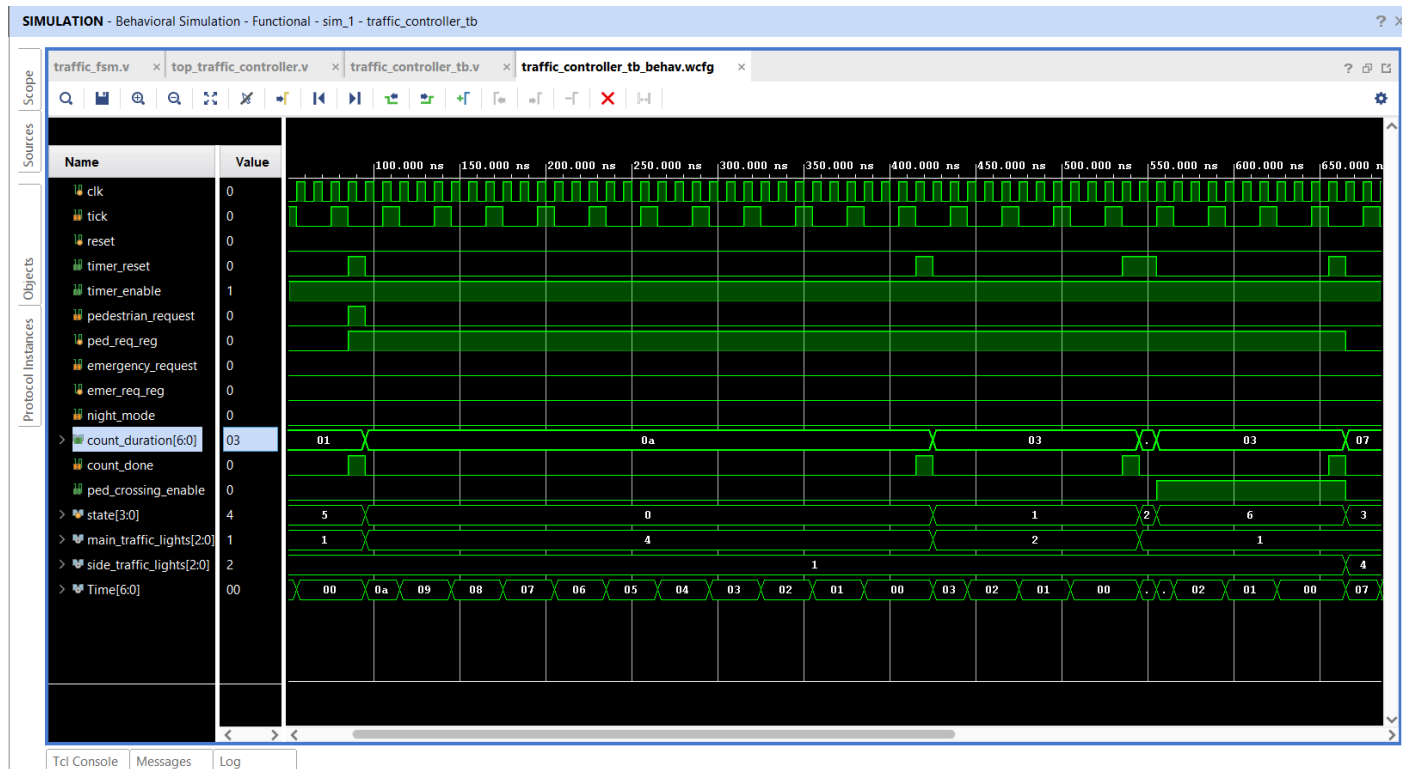


Figure 3.3 : Edge Condition Verification

TEST CASE 4 – EMERGENCY REQUEST DURING SIDE_GREEN

Objective: To verify emergency vehicle priority operation.

Procedure:

- Emergency request asserted during SIDE_GREEN

Expected Result:

- FSM transitions safely through yellow state
- Emergency state activated

Observed Result:

Normal traffic sequencing was interrupted safely for emergency handling .

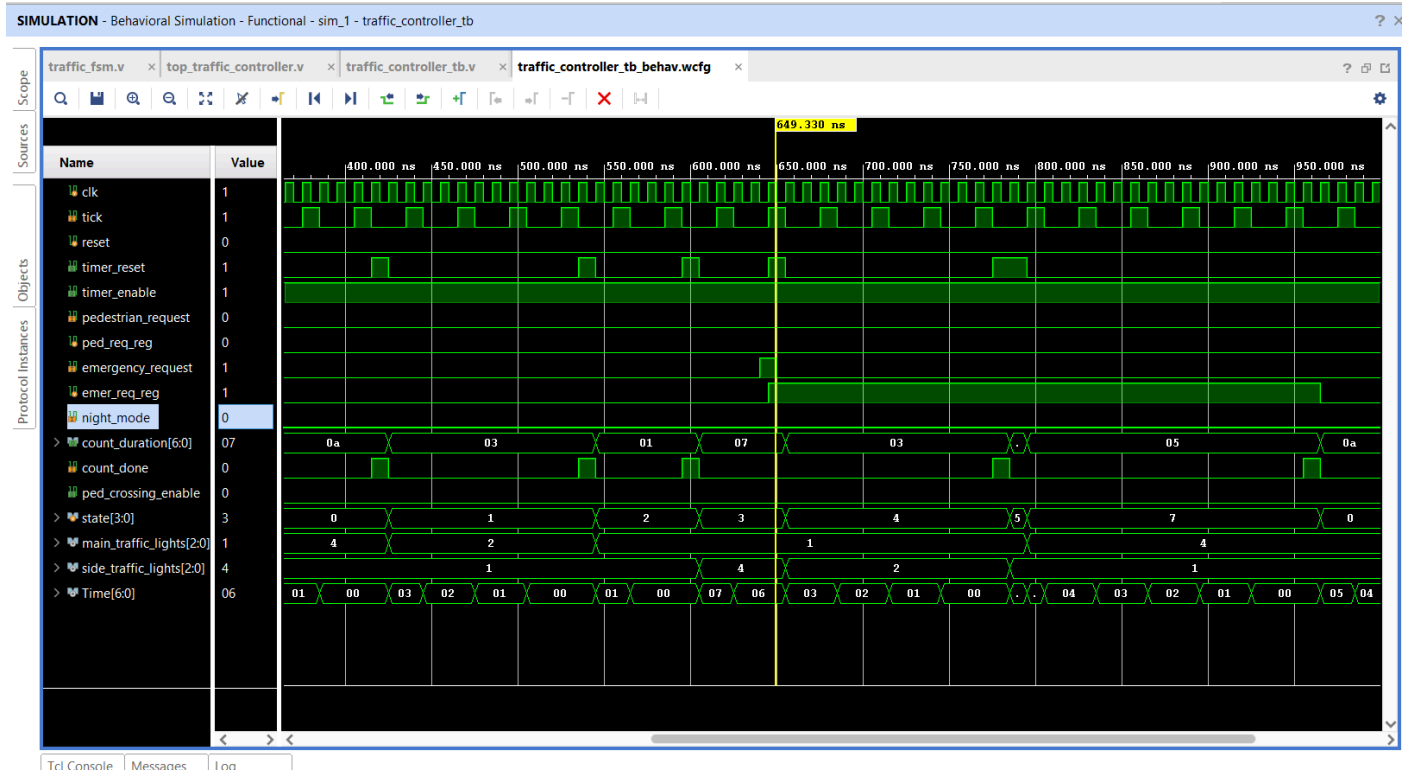


Figure 3.4 : Emergency Vehicle Override

TEST CASE 5 – EMERGENCY AND PEDESTRIAN REQUESTS TOGETHER

Objective: To verify request priority logic.

Procedure:

- Pedestrian and emergency requests asserted simultaneously

Expected Result:

- Emergency request receives higher priority
- Pedestrian request handled later

Observed Result:

Emergency handling was prioritized correctly while maintaining pedestrian request integrity.

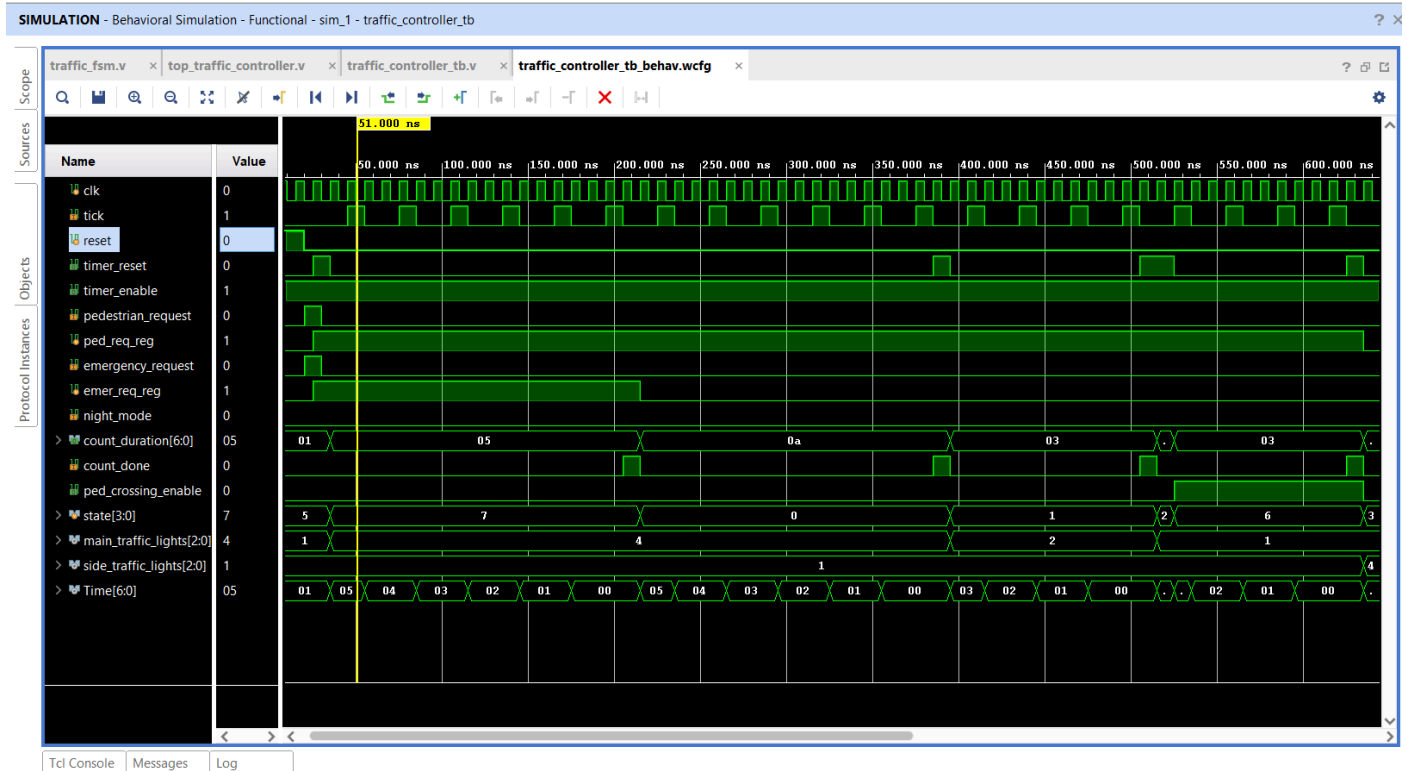


Figure 3.5 : Priority Handling Verification

TEST CASE 6 – NIGHT MODE ENABLE

Objective: To verify night mode blinking functionality.

Procedure:

- Night mode enabled during normal operation

Expected Result:

- FSM transitions to blinking states
- Yellow or Red blinking activated (Main Road - Yellow, Side Road - Red)

Observed Result:

Alternating blinking operation was observed as expected during night mode.

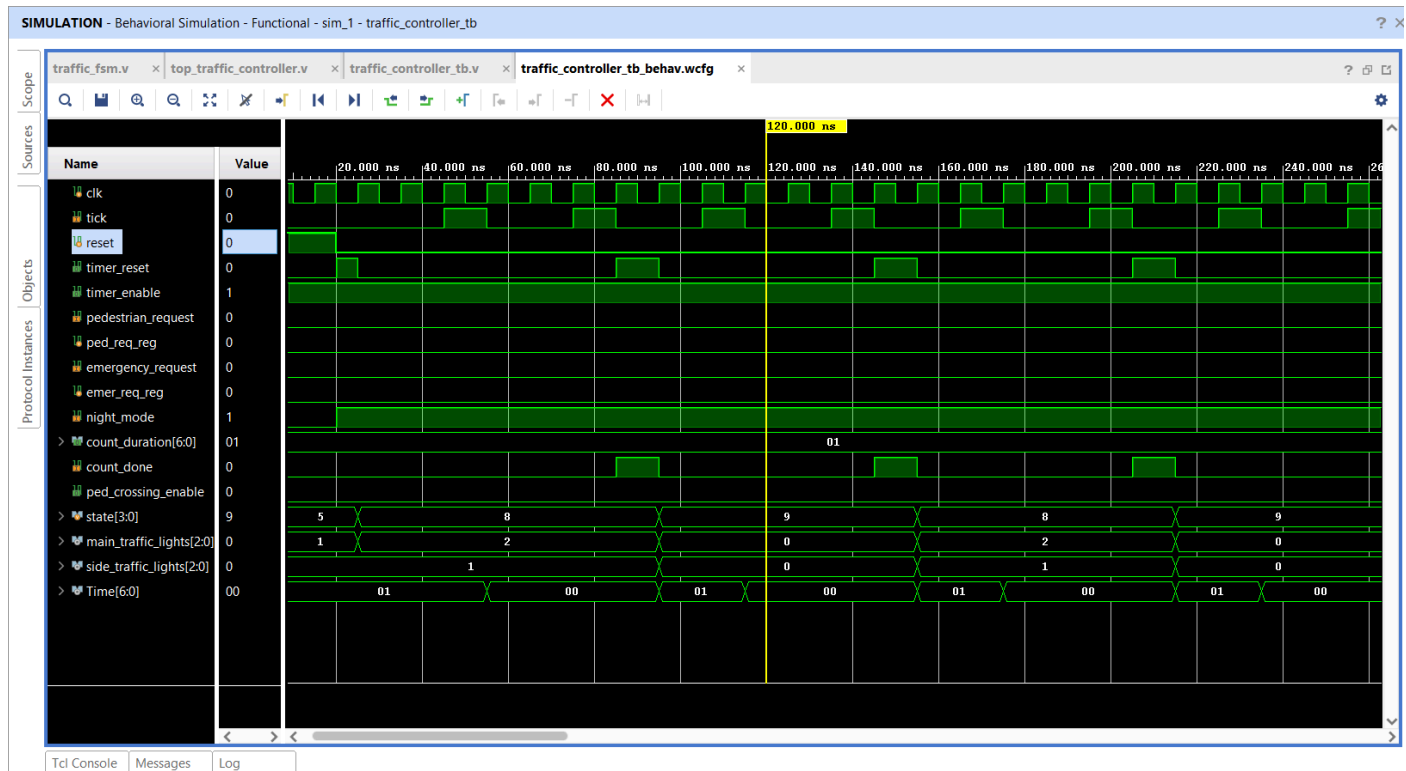


Figure 3.6 : Night Mode Operation

TEST CASE 7 – REQUESTS DURING NIGHT MODE

Objective: To verify handling of requests during night mode operation.

Procedure:

- Requests generated while night mode active

Expected Result:

- Requests ignored or deferred until night mode exit

Observed Result:

The FSM maintained stable blinking operation while safely preserving requests.

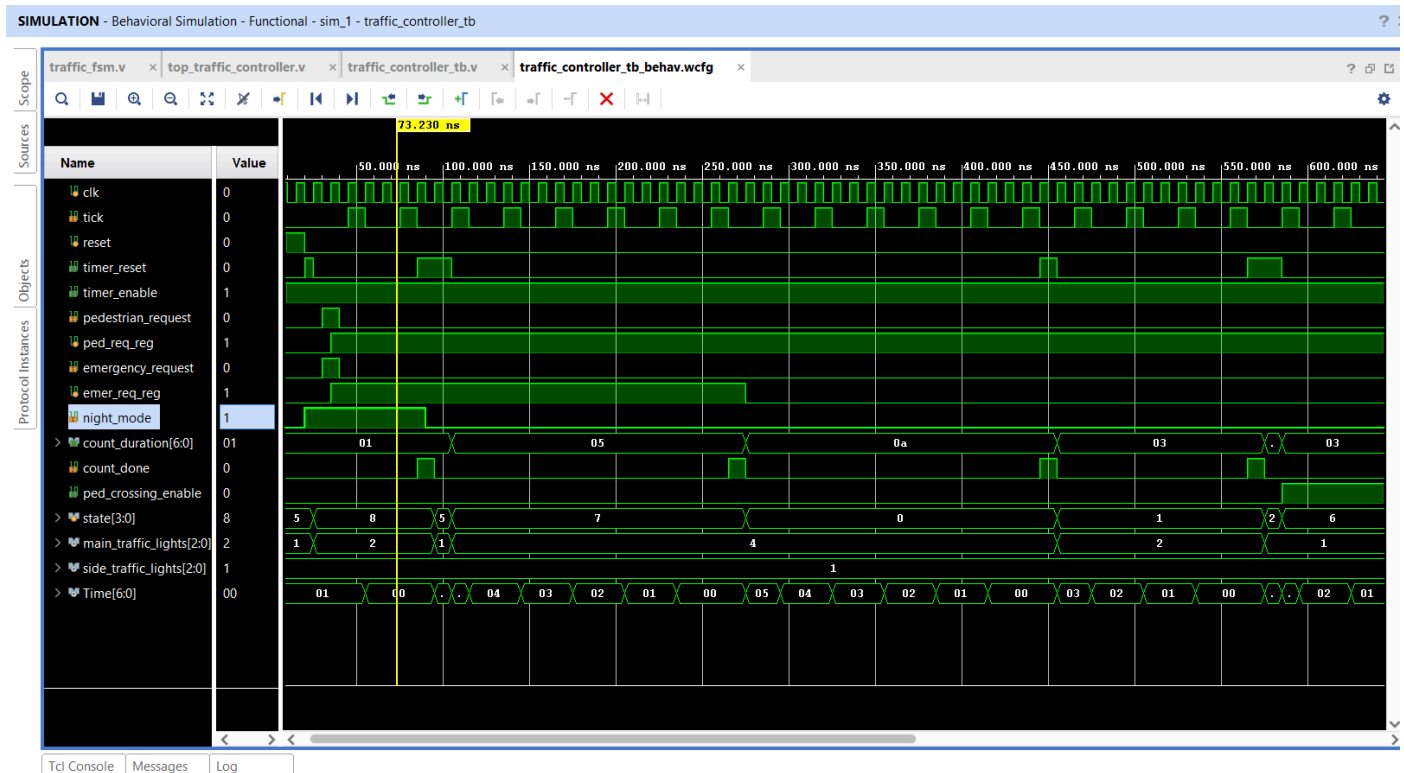


Figure 3.7 : Requests During Night Mode

TEST CASE 8 – EXIT NIGHT MODE

Objective: To verify restoration of previous traffic state.

Procedure:

- Night mode disabled after blinking operation

Expected Result:

- FSM returns to previous operational state

Observed Result:

Normal operation resumed correctly after night mode deactivation.

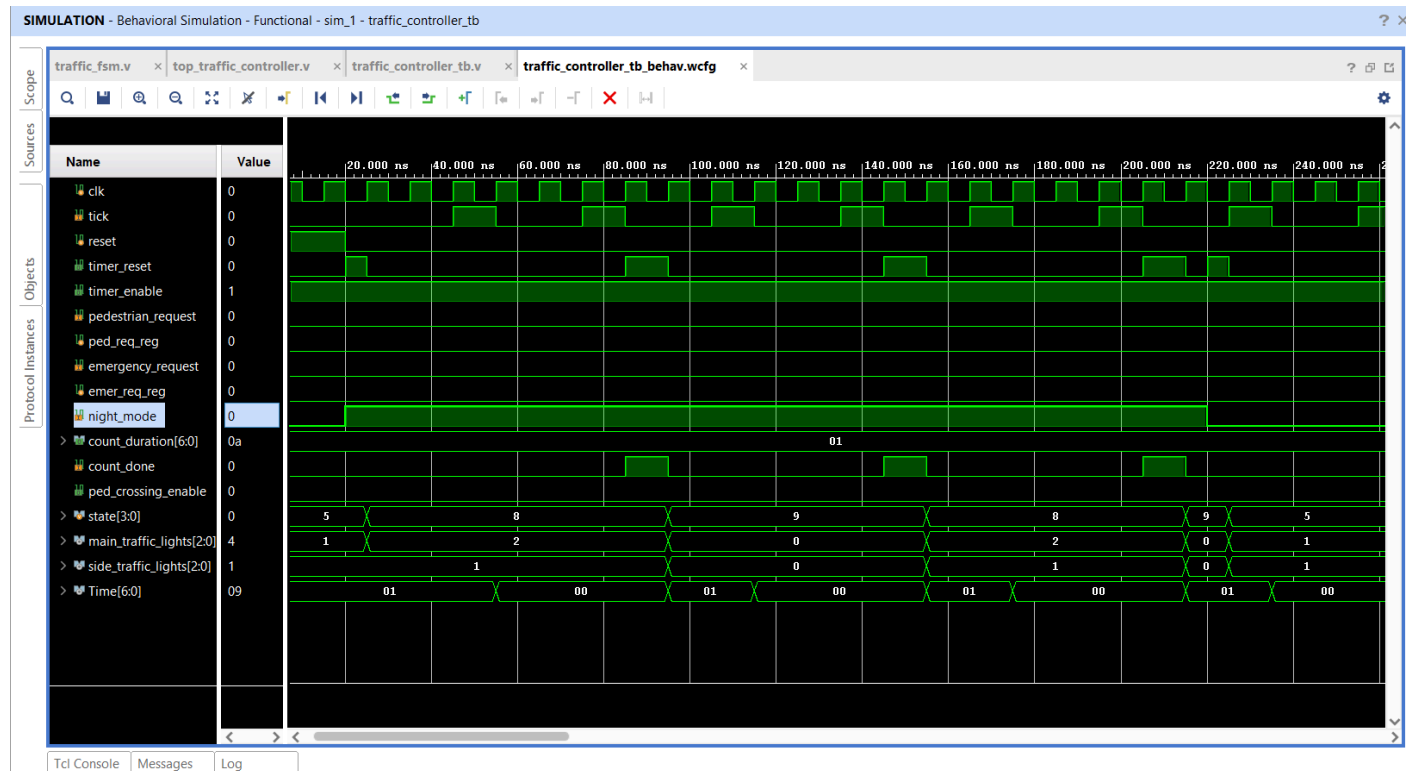


Figure 3.8 : Night Mode Exit Verification

TEST CASE 9 – RAPID NIGHT MODE TOGGLING

Objective: To verify FSM robustness against rapid control changes.

Procedure:

- Night mode toggled repeatedly

Expected Result:

- Stable FSM behavior without invalid transitions

Observed Result:

The FSM remained stable under rapid toggling conditions.

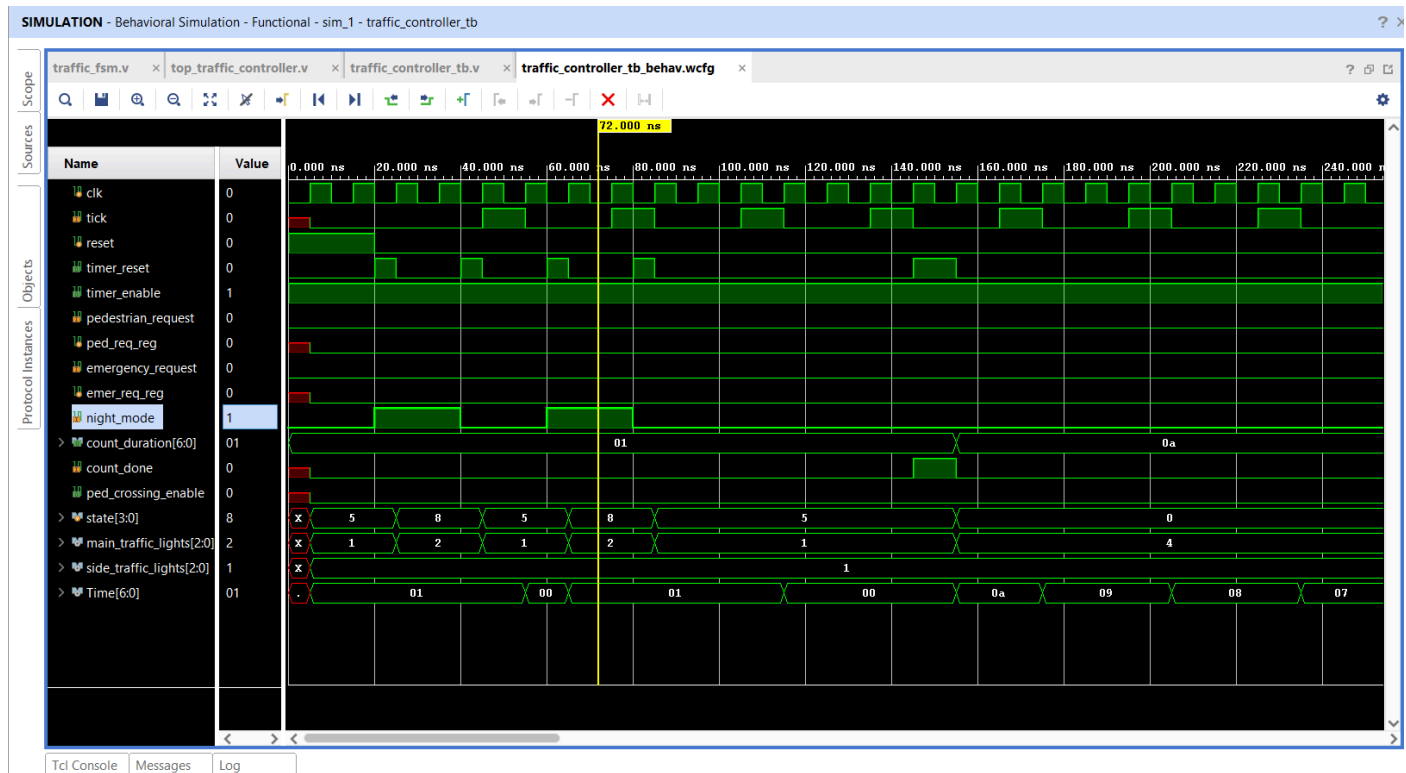


Figure 3.9 : Rapid Night Mode Toggle Verification

5. SIMULATION RESULTS SUMMARY

Waveform analysis confirmed the following observations:

- Correct FSM state sequencing
- Accurate timer operation
- Proper synchronization of requests
- Correct emergency prioritization
- Stable night mode blinking operation
- Reliable FSM transitions without glitches

The countdown timer output matched the configured state durations throughout the simulation.

Parameter	Status
FSM Transition Verification	Passed

Timer Operation	Passed
Pedestrian Handling	Passed
Emergency Handling	Passed
Night Mode Operation	Passed
Priority Logic	Passed
Edge Case Verification	Passed
Stable Outputs	Passed

6. CONCLUSION OF SIMULATION PHASE

The behavioral simulation successfully verified the complete functionality of the FPGA Based Smart Traffic Light Controller. All major operating conditions, including normal traffic sequencing, pedestrian servicing, emergency prioritization, and night mode blinking, operated correctly according to the design specifications.

The design was therefore considered functionally verified and ready for FPGA synthesis and implementation.

FPGA SYNTHESIS AND IMPLEMENTATION

After successful behavioral simulation and functional verification, the Smart Traffic Light Controller design was synthesized and implemented using Xilinx Vivado Design Suite targeting the Artix-7 FPGA.

The synthesis and implementation stages were used to verify hardware feasibility, FPGA resource utilization, timing performance, routing correctness, and successful bitstream generation.

The complete FPGA flow included:

- RTL Analysis
- Behavioral Simulation
- Synthesis
- Design Implementation
- Placement and Routing
- Design Rule Checking (DRC)
- Bitstream Generation

The final design successfully completed all stages without critical timing violations or bitstream generation failures.

1. SYNTHESIS OVERVIEW

Synthesis converts the Verilog HDL design into FPGA-compatible hardware logic.

The Smart Traffic Light Controller was synthesized successfully using Vivado synthesis tools.

The synthesis process verified:

- FSM logic correctness
- Register inference
- Combinational logic generation
- Counter implementation
- Parameterized module integration
- Hardware resource mapping

2. SYNTHESIS TOOL CONFIGURATION

Parameter	Value
-----------	-------

FPGA Tool	Xilinx Vivado 2024.x
Target FPGA	Artix-7 FPGA
HDL Language	Verilog HDL
Synthesis Strategy	Vivado Default
Clock Constraint	100 MHz
Top Module	top_traffic_controller

3. SYNTHESIS RESULTS

The synthesis process completed successfully without fatal errors. The generated RTL schematic confirmed correct interconnection between:

- traffic_fsm module
- timer_counter module
- clock_enable_generator module
- top_traffic_controller module

The synthesized design correctly inferred:

- Sequential registers
- FSM state registers
- Counters
- Combinational control logic
- Output decoding logic

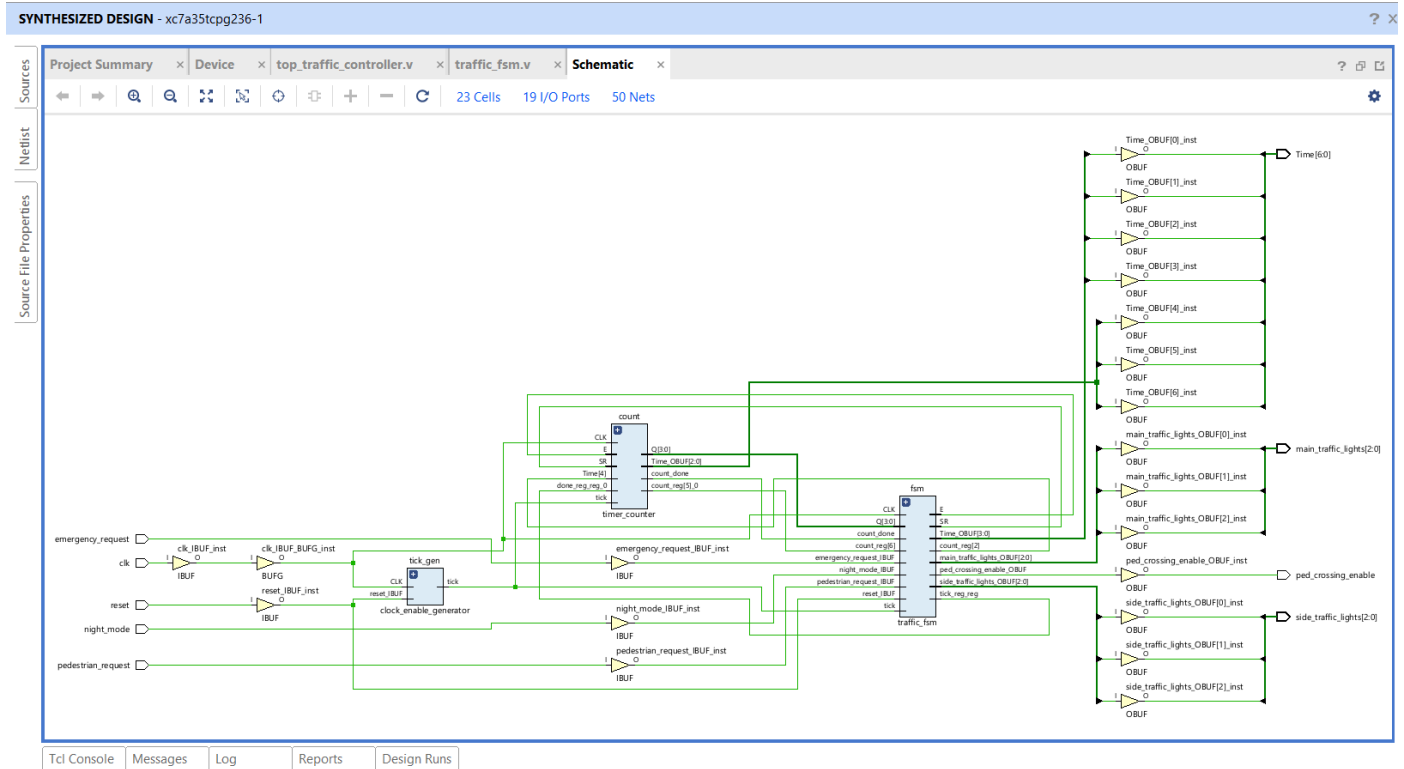


Figure 4 : RTL Schematic Generated After Synthesis

4. RESOURCE UTILIZATION SUMMARY

The synthesized design utilized only a small percentage of the available FPGA resources, making the design lightweight and suitable for future scalability.

8. Primitives

Ref Name	Used	Functional Category
FDRE	44	Flop & Latch
LUT6	24	LUT
LUT4	21	LUT
LUT5	18	LUT
OBUF	14	IO
LUT3	7	LUT
LUT2	7	LUT
CARRY4	7	CarryLogic
FDSE	6	Flop & Latch
IBUF	5	IO
LUT1	2	LUT
BUFG	1	Clock

Figure 5.1 : FPGA Resource Utilization Report Summary

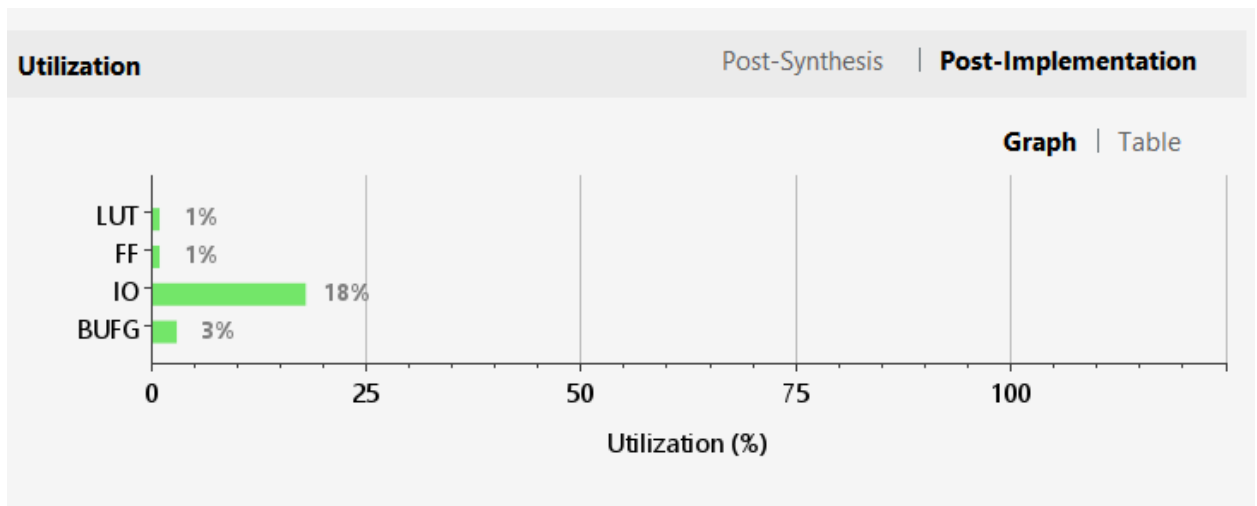


Figure 5.2 : FPGA Resource Utilization Graph

5. IMPLEMENTATION OVERVIEW

Implementation converts the synthesized netlist into a physically mapped FPGA design through:

- Translation
- Optimization
- Placement

- Routing

The implementation phase verifies whether the design can be physically mapped onto the FPGA while satisfying timing and routing constraints.

The Smart Traffic Light Controller completed implementation successfully without placement or routing failures.

6. DESIGN CONSTRAINTS

A complete Xilinx Design Constraints (XDC) file was developed to avoid common FPGA implementation issues such as:

- UCIO errors
- NSTD errors
- PLIO warnings
- Configuration voltage warnings

The constraints file included:

- Clock constraints
- Pin mapping constraints
- I/O standard assignments
- FPGA configuration voltage settings

The following FPGA properties were configured:

Property	Value
CFGBVS	VCCO
CONFIG_VOLTAGE	3.3 V
IOSTANDARD	LVCMOS33

The design constraints successfully eliminated all implementation blocking errors before bitstream generation.

```
#####
# FPGA Based Smart Traffic Light Controller
# Constraints File (.xdc)
# Target Device : Artix-7 FPGA
# Tool          : Vivado 2024.x
#####

set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

set_property PACKAGE_PIN W16 [get_ports {Time[0]}]
set_property PACKAGE_PIN V16 [get_ports {Time[1]}]
set_property PACKAGE_PIN W17 [get_ports {Time[2]}]
set_property PACKAGE_PIN V15 [get_ports {Time[3]}]
set_property PACKAGE_PIN W15 [get_ports {Time[4]}]
set_property PACKAGE_PIN P17 [get_ports {Time[5]}]
set_property PACKAGE_PIN R18 [get_ports {Time[6]}]

set_property IOSTANDARD LVCMOS33 [get_ports {Time[*]}]
```

Figure 6 : XDC Constraints File Configuration

7. PLACEMENT AND ROUTING RESULTS

Placement and routing were completed successfully by Vivado.

The implementation tools successfully:

- Allocated FPGA logic resources
- Connected internal routing paths
- Optimized signal timing
- Completed physical mapping

No critical routing congestion or implementation failures were observed.

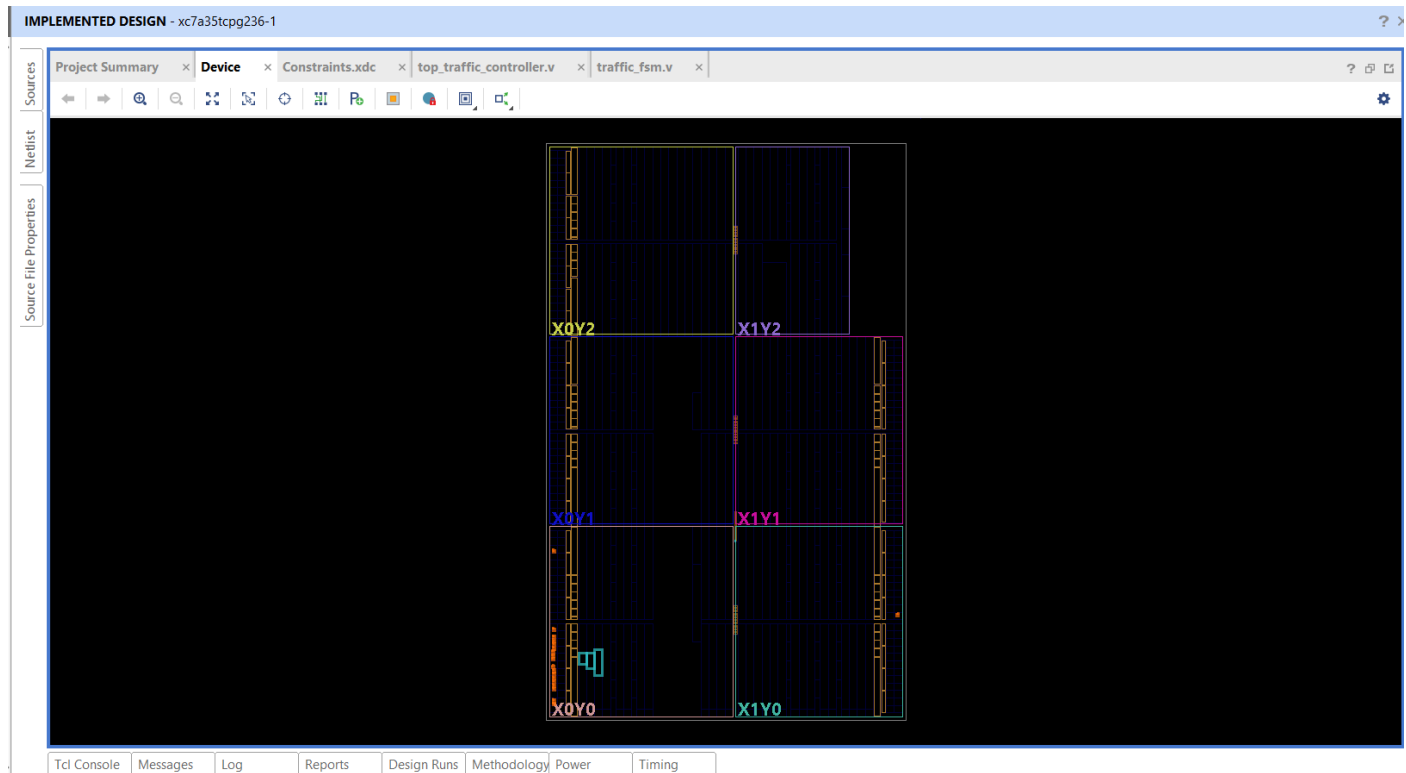


Figure 7 : Implemented Design View

8. TIMING ANALYSIS

Timing analysis was performed after implementation to verify correct clock-domain operation.

The design satisfied timing requirements successfully.

The timing report confirmed:

- No setup violations
- No hold violations
- Stable clock operation
- Successful timing closure

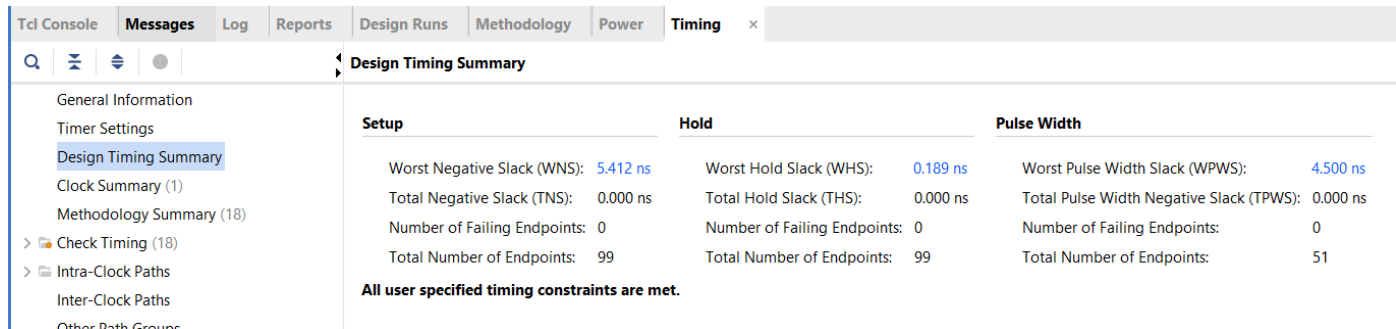


Figure 8 : Timing Summary Report

9. DESIGN RULE CHECK (DRC)

Vivado Design Rule Checking (DRC) was used to identify possible design or FPGA configuration issues before bitstream generation.

Initially observed warnings included:

- Missing LOC constraints
- Missing IOSTANDARD assignments
- Configuration voltage warnings

These issues were corrected by updating the XDC constraints file.

Final implementation completed without critical DRC violations.

```
Report DRC

Table of Contents
-----
1. REPORT SUMMARY
2. REPORT DETAILS

1. REPORT SUMMARY
-----
    Netlist: netlist
    Floorplan: checkpoint_top_traffic_controller
    Design limits: <entire design considered>
    Ruledeck: default
    Max checks: <unlimited>
    Checks found: 0

+-----+-----+-----+-----+
| Rule | Severity | Description | Checks |
+-----+-----+-----+-----+

2. REPORT DETAILS
-----
```


Figure 9 : DRC Verification Report

10. BITSTREAM GENERATION

After successful synthesis, implementation, routing, and timing verification, the FPGA bitstream was generated successfully.

The successful bitstream generation confirmed that:

- The design is hardware compatible
- FPGA mapping is valid
- Timing constraints are satisfied
- Physical implementation is complete

The generated bitstream can be directly programmed onto a compatible Artix-7 FPGA development board.

Tcl Console		Messages	Log	Reports	Design Runs		Methodology		Power	Timing										
Q		≡	⏮	⏪	⏩	⏭	+	%												
Name		Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	D	
✓ synth_1		constrs_1	synth_design Complete!												59	50	0	0		
✓ impl_1		constrs_1	write_bitstream Complete!	5.412	0.000	0.189	0.000		0.000	0.090	0	18 Warn			58	50	0	0		

Figure 10 : Successful Bitstream Generation

11. IMPLEMENTATION SUMMARY

Verification Parameter	Status
Synthesis	Passed
RTL Generation	Passed
Resource Utilization	Passed

Placement	Passed
Routing	Passed
Timing Closure	Passed
DRC Verification	Passed
Bitstream Generation	Passed

12. CONCLUSION OF FPGA IMPLEMENTATION

The FPGA implementation phase successfully verified the hardware feasibility of the Smart Traffic Light Controller design.

The complete Verilog HDL design was synthesized, implemented, routed, and converted into a valid FPGA bitstream without critical timing or hardware violations.

The lightweight resource utilization and successful timing closure demonstrate that the proposed design is efficient, scalable, and suitable for real-time FPGA-based traffic management applications.

ADVANTAGES OF THE PROPOSED SYSTEM

The FPGA Based Smart Traffic Light Controller provides several advantages over conventional fixed-timing traffic control systems.

The modular FSM-based architecture enables flexible traffic management, efficient hardware implementation, and real-time response capability.

Major advantages of the proposed system include:

- Real-time FPGA-based traffic control
- Emergency vehicle prioritization
- Dedicated pedestrian crossing support
- Low hardware resource utilization
- Modular and scalable architecture
- Reconfigurable timing parameters
- Reliable night mode operation
- Deterministic hardware execution

The use of Verilog HDL and FPGA implementation also allows the design to be extended for future intelligent traffic applications.

1. MODULAR DESIGN ADVANTAGE

The complete traffic controller was divided into multiple independent modules. The modular architecture improves:

- Design readability
- Debugging capability
- Reusability
- Scalability
- Verification efficiency

2. FPGA IMPLEMENTATION ADVANTAGES

Compared to microcontroller-based systems, FPGA implementation offers:

FPGA Advantages	Description
Parallel Processing	Simultaneous hardware execution
High Speed Operation	Faster response time
Reconfigurability	Easy logic modification

Low Latency	Hardware-level control
Scalability	Easy feature expansion
Deterministic Timing	Predictable real-time behavior

PRACTICAL DEPLOYMENT CONSIDERATIONS

Although the current implementation was verified primarily through FPGA simulation and implementation tools, the design architecture follows practical traffic control principles such as safe state transitions, emergency prioritization, and pedestrian servicing.

The timing values used during simulation were intentionally reduced to allow faster verification within manageable simulation time. During real hardware deployment, these timing parameters can be scaled using the programmable clock divider without modifying the FSM architecture.

This demonstrates the flexibility and scalability of the proposed FPGA-based traffic controller for future real-world implementation.

SAFETY IMPROVEMENTS

The proposed controller improves road safety through:

- Dedicated all-red safety states
- Controlled pedestrian crossings
- Emergency vehicle prioritization
- Stable night mode blinking operation
- Safe transition sequencing

These features reduce the possibility of unsafe traffic conflicts.

LIMITATIONS OF THE CURRENT SYSTEM

Although the proposed Smart Traffic Light Controller operates successfully, certain limitations exist in the current implementation.

The present design focuses primarily on FSM-based traffic management and behavioral verification.

Current limitations include:

- No adaptive timing control
- No real-time traffic optimization
- No hardware board testing performed
- Fixed traffic timing configuration

These limitations can be addressed in future enhancements of the project.

FUTURE SCOPE

The proposed FPGA Based Smart Traffic Light Controller can be further extended into a more intelligent and adaptive traffic management system.

Possible future enhancements include:

- Vehicle density detection using sensors
- Camera-based traffic monitoring
- AI-based traffic prediction
- Adaptive traffic timing algorithms
- IoT-enabled remote traffic control
- Wireless emergency vehicle communication
- Smart city infrastructure integration
- Real FPGA hardware deployment
- Mobile application monitoring
- Solar-powered traffic systems

The modular FPGA architecture makes the design suitable for future expansion without major hardware redesign.

APPLICATIONS

The FPGA Based Smart Traffic Light Controller can be applied in multiple real-world traffic management environments.

Applications include:

- Urban road intersections
- Highway traffic management
- Smart city infrastructure
- Emergency vehicle routing systems
- Pedestrian traffic control systems
- Educational FPGA learning platforms
- Research and embedded system laboratories

The design is also suitable for FPGA-based digital system education due to its modular FSM implementation and hardware verification flow.

DESIGN CHALLENGES AND LEARNING OUTCOMES

Several practical FPGA design challenges were encountered during the development of the Smart Traffic Light Controller.

These included:

- Proper synchronization of timer-controlled FSM transitions
- Safe handling of emergency and pedestrian requests
- Night mode state restoration
- FPGA timing and implementation warnings
- Constraint file configuration and pin mapping
- DRC error resolution during bitstream generation

Resolving these issues provided practical experience in Verilog HDL debugging, FPGA constraints management, hardware-oriented verification, and Vivado implementation flow.

The project also improved understanding of FSM-based digital system design and modular FPGA development methodology.

CONCLUSION

The FPGA Based Smart Traffic Light Controller was successfully designed, simulated, synthesized, implemented, and verified using Verilog HDL and Xilinx Vivado Design Suite.

The project successfully demonstrated:

- FSM-based traffic management
- Pedestrian request handling
- Emergency vehicle prioritization
- Night mode blinking operation
- Timer-controlled traffic sequencing
- FPGA hardware implementation flow

Behavioral simulation verified all major operating conditions, while FPGA synthesis and implementation confirmed hardware feasibility and successful bitstream generation.

The modular architecture, low hardware utilization, and successful timing closure demonstrate that the proposed system is efficient, scalable, and suitable for future intelligent traffic management applications.

This project also provided practical experience in:

- Verilog HDL design
- FSM development
- FPGA synthesis and implementation
- Xilinx Vivado workflow
- Digital hardware verification
- Constraint-based FPGA design

The successful completion of this project demonstrates the effectiveness of FPGA technology for real-time embedded traffic control systems.

REFERENCES

- Xilinx Inc., "Vivado Design Suite User Guide," Xilinx Documentation, 2024.
- Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd Edition, Pearson Education.
- FPGA4Student, "Traffic Light Controller using Verilog HDL," Online FPGA Learning Resource.
- Xilinx Artix-7 FPGA Datasheet, Xilinx Inc.
- IEEE Research Papers on Intelligent Traffic Management Systems.